Abstract Classes and Interfaces

Module-2



Dr.M. Sivakumar

Professor & Technical Trainer / CSE

Saveetha School of Engineering, SIMATS

sivakumarm.sse@saveetha.com

+91-9500600868

Abstract Class

- In any programming language, a class which is declared with the keyword abstract is known as an abstract class.
- An abstract class captures common characteristics of subclasses and may or may not contain any abstract method.
- It cannot be instantiated but can be only used as a superclass by its subclasses.

```
1.   abstract class <class-name>
2.   {
3.    //class definition
4.   }
```

Abstract Class

key points

- An abstract class can have constructors and static methods
- It can have final methods, they force the subclass not to change the body of the method
- You can use an abstract class by inheriting it from another class and then provide implementations to the abstract methods in it
- If an abstract class doesn't have any method implementation, it's always better to use interface
- A class declared abstract may or may not include abstract methods

Abstract Class

Abstract method

- An abstract method is one which contains only declaration or prototype but it never contains body or definition.
- In order to make any undefined method as abstract whose declaration is must be predefined by abstract keyword.

Syntax

```
abstract ReturnType methodName(List of formal parameter)
```

Abstract Class

Key Points about abstract method

- Abstract methods don't have an implementation (body), they just have method signature as shown in the above example
- If a class has an abstract method it should be declared abstract, the vice versa is not true
- Instead of curly braces, an abstract method will have a semicolon (;) at the end
- If a regular class extends an abstract class, then the class must have to implement all the abstract methods of that class or it has to be declared abstract as well

Abstract Class

Example:

```
//abstract parent class
abstract class Animal
{
 //abstract method
    public abstract void sound();
 }
```

```
//Dog class extends Animal class
public class Dog extends Animal
{
      public void sound()
      {
      System.out.println("Woof");
      }
public static void main(String args[])
{
      Animal obj = new Dog();
      obj.sound();
 }
 }
```

Abstract Class

Rules

- A class derived from the abstract class must implement all those methods that are declared as abstract in the parent class.
- Abstract class cannot be instantiated which means you cannot create the object of it.
- If a child does not implement all the abstract methods of abstract parent class, then the child class must need to be declared abstract as well.

Abstract Class

Advantage of abstract class

- ✓ Less memory space for the application
- ✓ Less execution time
- ✓ More performance

Interface

- ***An interface in Java is defined as an abstract type that specifies class behavior.***
- It is 100% impure class
- Interfaces are syntactically similar to classes, but you cannot create instance of an **Interface**
- By interface, we can support the functionality of multiple inheritance
- It can be used to achieve loose coupling

Syntax

```
modifier interface interfaceName {
    //constant declarations
    //method signatures
}
```

Interface

Key Points

- Methods inside interface must not be static and final
- All variables declared inside interface are implicitly public, static and final.
- All methods declared inside interfaces are implicitly public and abstract, even if you don't use public or abstract keyword.
- Interface can extend one or more other interface.
- Interface cannot implement a class.
- Interface can be nested inside another interface.

Interface

Example:

```
interface Person
{
void run();  // abstract method
}
class A implements Person
{
public void run()
{
System.out.println("Run fast");
}
public static void main(String args[])
 {
 A obj = new A();
 obj.run();
 }
}
```

Output

```
Run fast
```

Interface

Multiple inheritance using interface

```
interface Developer
{
void disp();
}
interface Manager
{
void show();
}

class Employee implements Developer, Manager
{
public void disp()
{
System.out.println("Hello Good Morning");
}
```

```
public void show()
{
System.out.println("How are you ?");
}
public static void main(String args[])
{
Employee obj=new Employee();
obj.disp();
obj.show();
}
}
```

Output

```
Hello Good Morning
How are you ?
```

Interface

Interface extends another interface

```
interface PersonalInfo
{
    //Body of Interface
}

interface CollegeInfo extends PersonalInfo
{
    //Body of Interface
}
```

Interface extends multiple interfaces

```
interface PersonalInfo
{
    //Body of Interface
}

interface CollegeInfo
{
    //Body of Interface
}

interface Student extends PersonalInfo, CollegeInfo
{
    //Body of Interface
}
```

Interface Vs Abstract Classes

Abstract Classes		Interfaces
Abstract classes are fast.	Speed	Interfaces are slow.
Abstract classes can extend only one class.	Multiple Inheritance	Interface can implement several interfaces.
You can define fields as well as constants.	Defined Fields	You cannot define fields in an interface.
A single abstract class can extend one & only one interface.	Extension Limit	A single interface can extend multiple interfaces.